

✓ OpsPilot AI: Intelligent Shipment Delay Risk Prediction

Project Overview

OpsPilot AI is a machine learning solution designed to predict whether a shipment is at risk of delayed delivery before dispatch. By identifying high-risk orders early, logistics teams can prioritize shipments, reduce operational delays, and improve customer satisfaction.

Dataset

- Source: DataCo Smart Supply Chain Dataset (Kaggle)
- Records: 180,519 shipment records
- Features: 53 supply chain, customer, order, product, and shipping attributes
- Target Variable: `Late_delivery_risk`

✓ 1. Dataset Exploration

Load the supply chain datasets and inspect their structure, features, and data quality to identify the most suitable dataset for delay prediction.

```
# Install if needed
!pip -q install kagglehub

import kagglehub
import pandas as pd
import os
from glob import glob

# Download latest version
path = kagglehub.dataset_download("shashwatwork/dataco-smart-supply-chain-for-big-data-analysis")

print("="*70)
print("Dataset Folder:")
print(path)

print("\n" + "="*70)
print("Files:")
for file in os.listdir(path):
    print(file)

# Find all CSV files
csv_files = glob(os.path.join(path, "*.csv"))

print("\n" + "="*70)
print(f"CSV Files Found: {len(csv_files)}")

datasets = {}

for csv in csv_files:
    name = os.path.basename(csv).replace(".csv", "")
    df = pd.read_csv(csv, encoding="latin1")

    datasets[name] = df

    print("\n" + "="*70)
    print(f"Dataset: {name}")
    print("="*70)

    print(f"Shape: {df.shape}")

    print("\nColumns:")
    for col in df.columns:
        print("-", col)

    print("\nData Types:")
    print(df.dtypes)

    print("\nMissing Values:")
    print(df.isnull().sum())

    print("\nFirst 5 Rows:")
    display(df.head())
```

```
print("\n" + "="*70)
print("Loaded datasets:", list(datasets.keys()))
```


Using Colab cache for faster access to the 'dataco-smart-supply-chain-for-big-data-analysis' dataset.

Dataset Folder:
/kaggle/input/dataco-smart-supply-chain-for-big-data-analysis

Files:
tokenized_access_logs.csv
DataCoSupplyChainDataset.csv
DescriptionDataCoSupplyChain.csv

CSV Files Found: 3

Dataset: tokenized_access_logs

Shape: (469977, 8)

Columns:
- Product
- Category
- Date
- Month
- Hour
- Department
- ip
- url

Data Types:
Product object
Category object
Date object
Month object
Hour int64
Department object
ip object
url object
dtype: object

Missing Values:
Product 0
Category 0
Date 0
Month 0
Hour 0
Department 0
ip 0
url 0
dtype: int64

First 5 Rows:

	Product	Category	Date	Month	Hour	Department	ip	url
0	adidas Brazuca 2017 Official Match Ball	baseball & softball	9/1/2017 6:00	Sep	6	fitness	37.97.182.65	/department/fitness/category/baseball%20&%20so...
1	The North Face Women's Recon Backpack	hunting & shooting	9/1/2017 6:00	Sep	6	fan shop	206.56.112.1	/department/fan%20shop/category/hunting%20&%20...
2	adidas Kids' RG III Mid Football Cleat	featured shops	9/1/2017 6:00	Sep	6	apparel	215.143.180.0	/department/apparel/category/featured%20shops/...
3	Under Armour Men's Compression EV SL Slide	electronics	9/1/2017 6:00	Sep	6	footwear	206.56.112.1	/department/footwear/category/electronics/prod...
4	Pelican Sunstream 100 Kayak	water sports	9/1/2017 6:01	Sep	6	fan shop	136.108.56.242	/department/fan%20shop/category/water%20sports...

Dataset: DataCoSupplyChainDataset

Shape: (180519, 53)

Columns:
- Type
- Days for shipping (real)
- Days for shipment (scheduled)
- Benefit per order
- Sales per customer
- Delivery Status
- Late_delivery_risk

- Category Id
- Category Name
- Customer City
- Customer Country
- Customer Email
- Customer Fname
- Customer Id
- Customer Lname
- Customer Password
- Customer Segment
- Customer State
- Customer Street
- Customer Zipcode
- Department Id
- Department Name
- Latitude
- Longitude
- Market
- Order City
- Order Country
- Order Customer Id
- order date (DateOrders)
- Order Id
- Order Item Cardprod Id
- Order Item Discount
- Order Item Discount Rate
- Order Item Id
- Order Item Product Price
- Order Item Profit Ratio
- Order Item Quantity
- Sales
- Order Item Total
- Order Profit Per Order
- Order Region
- Order State
- Order Status
- Order Zipcode
- Product Card Id
- Product Category Id
- Product Description
- Product Image
- Product Name
- Product Price
- Product Status
- shipping date (DateOrders)
- Shipping Mode

Data Types:

Type	object
Days for shipping (real)	int64
Days for shipment (scheduled)	int64
Benefit per order	float64
Sales per customer	float64
Delivery Status	object
Late_delivery_risk	int64
Category Id	int64
Category Name	object
Customer City	object
Customer Country	object
Customer Email	object
Customer Fname	object
Customer Id	int64
Customer Lname	object
Customer Password	object
Customer Segment	object
Customer State	object
Customer Street	object
Customer Zipcode	float64
Department Id	int64
Department Name	object
Latitude	float64
Longitude	float64
Market	object
Order City	object
Order Country	object
Order Customer Id	int64
order date (DateOrders)	object
Order Id	int64
Order Item Cardprod Id	int64
Order Item Discount	float64
Order Item Discount Rate	float64
Order Item Id	int64
Order Item Product Price	float64
Order Item Profit Ratio	float64
Order Item Quantity	int64
Sales	float64
Order Item Total	float64
Order Profit Per Order	float64
Order Region	object
Order State	object
Order Status	object

```

-----
Order Zipcode          float64
Product Card Id        int64
Product Category Id    int64
Product Description     float64
Product Image          object
Product Name           object
Product Price          float64
Product Status         int64
shipping date (DateOrders) object
Shipping Mode          object
dtype: object

```

Missing Values:

```

Type          0
Days for shipping (real) 0
Days for shipment (scheduled) 0
Benefit per order 0
Sales per customer 0
Delivery Status 0
Late_delivery_risk 0
Category Id    0
Category Name  0
Customer City  0
Customer Country 0
Customer Email 0
Customer Fname 0
Customer Id     0
Customer Lname  8
Customer Password 0
Customer Segment 0
Customer State  0
Customer Street 0
Customer Zipcode 3
Department Id   0
Department Name 0
Latitude        0
Longitude       0
Market          0
Order City      0
Order Country   0
Order Customer Id 0
order date (DateOrders) 0
Order Id        0
Order Item Cardprod Id 0
Order Item Discount 0
Order Item Discount Rate 0
Order Item Id   0
Order Item Product Price 0
Order Item Profit Ratio 0
Order Item Quantity 0
Sales           0
Order Item Total 0
Order Profit Per Order 0
Order Region    0
Order State     0
Order Status    0
Order Zipcode   155679
Product Card Id 0
Product Category Id 0
Product Description 180519
Product Image    0
Product Name     0
Product Price    0
Product Status   0
shipping date (DateOrders) 0
Shipping Mode     0
dtype: int64

```

First 5 Rows:

	Type	Days for shipping (real)	Days for shipment (scheduled)	Benefit per order	Sales per customer	Delivery Status	Late_delivery_risk	Category Id	Category Name	Customer City	...
0	DEBIT	3	4	91.250000	314.640015	Advance shipping	0	73	Sporting Goods	Caguas	...
1	TRANSFER	5	4	-249.089996	311.359985	Late delivery	1	73	Sporting Goods	Caguas	...
2	CASH	4	4	-247.779999	309.720001	Shipping on time	0	73	Sporting Goods	San Jose	...
3	DEBIT	3	4	22.860001	304.809998	Advance shipping	0	73	Sporting Goods	Los Angeles	...
4	PAYMENT	2	4	134.210007	298.250000	Advance shipping	0	73	Sporting Goods	Caguas	...

5 rows × 53 columns

Dataset: DescriptionDataCoSupplyChain

2. Data Preparation

Shape: (52, 2)

Clean the raw data by removing irrelevant attributes, preventing target leakage, handling missing values, and preparing a reliable dataset for modeling.

- DESCRIPTION

```
# =====
# OPSPILOT AI
# PHASE 1 - DATA PREPARATION
# =====

import pandas as pd
import numpy as np
import joblib
from sklearn.preprocessing import LabelEncoder

# =====
# LOAD DATASET
# =====

df = datasets["DataCoSupplyChainDataset"].copy()

print("="*80)
print("Original Shape :", df.shape)

# =====
# TARGET
# =====

TARGET = "Late_delivery_risk"

# =====
# REMOVE TARGET LEAKAGE
# =====

leakage_cols = [
    "Delivery Status",
    "Days for shipping (real)"
]

df.drop(columns=[c for c in leakage_cols if c in df.columns],
        inplace=True)

# =====
# REMOVE USELESS COLUMNS
# =====

drop_cols = [

    # IDs
    "Customer Id",
    "Order Customer Id",
    "Order Id",
    "Category Id",
    "Department Id",
    "Product Card Id",
    "Product Category Id",
    "Order Item Id",
    "Order Item Cardprod Id",

    # Personal Info
    "Customer Email",
    "Customer Password",
    "Customer Fname",
    "Customer Lname",
    "Customer Street",

    # Empty / Mostly Missing
    "Product Description",
    "Order Zipcode",
    "Customer Zipcode",

    # Images
    "Product Image",

    # Coordinates
    "Latitude",
    "Longitude"
]
```

```

df.drop(columns=[c for c in drop_cols if c in df.columns],
         inplace=True)

print("After Dropping :", df.shape)

# =====
# DATE FEATURES
# =====

df["order date (DateOrders)"] = pd.to_datetime(
    df["order date (DateOrders)"],
    errors="coerce"
)

df["shipping date (DateOrders)"] = pd.to_datetime(
    df["shipping date (DateOrders)"],
    errors="coerce"
)

df["Order Month"] = df["order date (DateOrders)"].dt.month

df["Order Weekday"] = df["order date (DateOrders)"].dt.dayofweek

df["Order Quarter"] = df["order date (DateOrders)"].dt.quarter

# Days between order and shipping

df["Processing Days"] = (
    df["shipping date (DateOrders)"] -
    df["order date (DateOrders)"]
).dt.days

# Remove original dates

df.drop(columns=[
    "order date (DateOrders)",
    "shipping date (DateOrders)"
], inplace=True)

# =====
# FEATURE ENGINEERING
# =====

# Profit Margin

df["Profit Margin"] = (
    df["Order Profit Per Order"] /
    (df["Sales"] + 1)
)

# Discount Amount

df["Discount Amount"] = (
    df["Order Item Discount"] *
    df["Order Item Quantity"]
)

# Average Selling Price

df["Average Selling Price"] = (
    df["Sales"] /
    (df["Order Item Quantity"] + 1)
)

# High Value Order

median_sales = df["Sales"].median()

df["High Value Order"] = (
    df["Sales"] > median_sales
).astype(int)

# =====
# REMOVE HIGH CARDINALITY
# =====

# Product Name has thousands of unique values.
# Remove it for a cleaner baseline model.

if "Product Name" in df.columns:
    df.drop(columns="Product Name", inplace=True)

```



```
# =====
# HANDLE MISSING VALUES
# =====

print("\nMissing Values Before Cleaning")
print(df.isnull().sum().sort_values(ascending=False).head(10))

df.dropna(inplace=True)

print("\nAfter Removing Missing :", df.shape)

# =====
# ENCODE CATEGORICAL VARIABLES
# =====

encoders = {}

categorical_cols = df.select_dtypes(include="object").columns

print("\nEncoding Columns:")

for col in categorical_cols:

    print(".", col)

    le = LabelEncoder()

    df[col] = le.fit_transform(df[col].astype(str))

    encoders[col] = le

# Save encoders

joblib.dump(encoders, "label_encoders.pkl")

# =====
# FINAL DATASET
# =====

print("\nFinal Shape :", df.shape)

print("\nTarget Distribution")

print(df[TARGET].value_counts())

print("\nColumns")

print(df.columns.tolist())

display(df.head())

# Save

df.to_csv("clean_supply_chain.csv", index=False)

print("\nSaved:")
print("clean_supply_chain.csv")
print("label_encoders.pkl")

print("\n✅ Phase 1 Complete")
```

```
=====
Original Shape : (180519, 53)
After Dropping : (180519, 31)

Missing Values Before Cleaning
Type                                0
Days for shipment (scheduled)      0
Benefit per order                  0
Sales per customer                 0
Late_delivery_risk                 0
Category Name                      0
Customer City                     0
Customer Country                  0
Customer Segment                  0
Customer State                    0
dtype: int64

After Removing Missing : (180519, 36)

Encoding Columns:
• Type
• Category Name
• Customer City
• Customer Country
• Customer Segment
• Customer State
• Department Name
• Market
• Order City
• Order Country
• Order Region
• Order State
• Order Status
• Shipping Mode

Final Shape : (180519, 36)

Target Distribution
Late_delivery_risk
1    98977
0   81542
Name: count, dtype: int64

Columns
['Type', 'Days for shipment (scheduled)', 'Benefit per order', 'Sales per customer', 'Late_delivery_risk', 'Category Name',
Type      Days for shipment (scheduled)  Benefit per order  Sales per customer  Late_delivery_risk  Category Name  Customer City  Customer Country  Customer Segment  Customer State  ...  Prod Sta
0      1              4    91.250000    314.640015              0          40          66              1              0              36  ...
1      3              4   -249.089996    311.359985              1          40          66              1              0              36  ...
2      0              4   -247.779999    309.720001              0          40         452              0              0              5  ...
3      1              4    22.860001    304.809998              0          40         285              0              2              5  ...
4      2              4   134.210007    298.250000              0          40          66              1              1              36  ...

5 rows x 36 columns

Saved:
clean_supply_chain.csv
label_encoders.pkl

✅ Phase 1 Complete
```

3. Baseline Model Development

Train an initial CatBoost classifier to establish a strong baseline and evaluate its ability to predict shipment delay risk.

```
!pip -q install catboost
```

97.1/97.1 MB 5.2 MB/s eta 0:00:00

4. Model Validation

The initial performance appeared unusually high. Investigate whether certain features contain hidden information that could leak the target.

```

# =====
# CATBOOST EXPERIMENT (ONE CELL)
# =====

import pandas as pd
import numpy as np
import joblib

from catboost import CatBoostClassifier

from sklearn.model_selection import train_test_split

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    classification_report,
    confusion_matrix
)

# -----
# Load ORIGINAL dataframe
# -----

df_cat = datasets["DataCoSupplyChainDataset"].copy()

print("Original Shape:", df_cat.shape)

# -----
# Drop columns
# -----

drop_cols = [
    "Customer Password",
    "Customer Email",
    "Customer Fname",
    "Customer Lname",
    "Customer Street",
    "Customer Zipcode",
    "Product Image",
    "Product Description",
    "Order Zipcode",
    "Latitude",
    "Longitude",
    "Order Customer Id",
    "Customer Id",
    "Category Id",
    "Department Id",
    "Product Card Id",
    "Product Category Id",
    "Order Item Id",
    "Order Item Cardprod Id",
    "Order Id",
    "Delivery Status",
    "Days for shipping (real)"
]

df_cat.drop(
    columns=[c for c in drop_cols if c in df_cat.columns],
    inplace=True
)

# -----
# Dates
# -----

df_cat["order date (DateOrders)"] = pd.to_datetime(
    df_cat["order date (DateOrders)"],
    errors="coerce"
)

df_cat["Order Month"] = df_cat["order date (DateOrders)"].dt.month
df_cat["Order Weekday"] = df_cat["order date (DateOrders)"].dt.dayofweek
df_cat["Order Quarter"] = df_cat["order date (DateOrders)"].dt.quarter

# -----
# Features
# -----

df_cat["Profit Margin"] = (

```

```

        df_cat["Order Profit Per Order"] /
        (df_cat["Sales"] + 1)
    )

    df_cat["Discount Amount"] = (
        df_cat["Order Item Product Price"] *
        df_cat["Order Item Discount Rate"]
    )

    df_cat["Average Selling Price"] = (
        df_cat["Sales"] /
        df_cat["Order Item Quantity"].replace(0,1)
    )

    median_sales = df_cat["Sales"].median()

    df_cat["High Value Order"] = (
        df_cat["Sales"] > median_sales
    ).astype(int)

    df_cat.drop(
        columns=[
            "order date (DateOrders)",
            "shipping date (DateOrders)"
        ],
        inplace=True
    )

    df_cat.dropna(inplace=True)

    print("After Cleaning:", df_cat.shape)

# -----
# Features
# -----

target = "Late_delivery_risk"

X = df_cat.drop(columns=[target])

y = df_cat[target]

# categorical columns

cat_features = X.select_dtypes(include=["object"]).columns.tolist()

cat_index = [
    X.columns.get_loc(col)
    for col in cat_features
]

print("\nCategorical Features:")
print(cat_features)

# -----
# Split
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    stratify=y,
    random_state=42
)

# -----
# Model
# -----

model = CatBoostClassifier(

    iterations=1000,

    depth=8,

    learning_rate=0.05,

    loss_function="Logloss",

    eval_metric="AUC",

```

```

        random_seed=42,

        verbose=100
    )

print("\nTraining...\n")

model.fit(

    X_train,
    y_train,

    cat_features=cat_index,

    eval_set=(X_test,y_test),

    use_best_model=True
)

# -----
# Prediction
# -----

y_prob = model.predict_proba(X_test)[: ,1]

y_pred = (y_prob>=0.40).astype(int)

# -----
# Metrics
# -----

print("\n"+"="*70)

print(f"Accuracy : {accuracy_score(y_test,y_pred)*100:.2f}%")

print(f"Precision : {precision_score(y_test,y_pred)*100:.2f}%")

print(f"Recall : {recall_score(y_test,y_pred)*100:.2f}%")

print(f"F1 Score : {f1_score(y_test,y_pred)*100:.2f}%")

print(f"ROC AUC : {roc_auc_score(y_test,y_prob):.4f}")

print("="*70)

print("\nConfusion Matrix\n")

print(confusion_matrix(y_test,y_pred))

print("\nClassification Report\n")

print(classification_report(y_test,y_pred))

joblib.dump(model,"catboost_model.pkl")

print("\nModel Saved : catboost_model.pkl")

```

Original Shape: (180519, 53)
 After Cleaning: (180519, 36)

Categorical Features:

['Type', 'Category Name', 'Customer City', 'Customer Country', 'Customer Segment', 'Customer State', 'Department Name', 'Mar

Training...

0:	test: 0.7699784	best: 0.7699784 (0)	total: 1.54s	remaining: 25m 39s
100:	test: 0.9815783	best: 0.9815799 (98)	total: 1m 18s	remaining: 11m 40s
200:	test: 0.9821686	best: 0.9821686 (200)	total: 2m 34s	remaining: 10m 13s
300:	test: 0.9825609	best: 0.9825669 (296)	total: 3m 48s	remaining: 8m 50s
400:	test: 0.9827564	best: 0.9827565 (397)	total: 5m 5s	remaining: 7m 36s
500:	test: 0.9828877	best: 0.9829005 (488)	total: 6m 22s	remaining: 6m 20s
600:	test: 0.9830547	best: 0.9830586 (599)	total: 7m 38s	remaining: 5m 4s
700:	test: 0.9831178	best: 0.9831278 (693)	total: 8m 55s	remaining: 3m 48s
800:	test: 0.9832111	best: 0.9832125 (799)	total: 10m 13s	remaining: 2m 32s
900:	test: 0.9833871	best: 0.9833907 (896)	total: 11m 31s	remaining: 1m 15s
999:	test: 0.9834671	best: 0.9834671 (999)	total: 12m 47s	remaining: 0us

bestTest = 0.9834670736
 bestIteration = 999

=====
 Accuracy : 91.20%
 Precision : 89.92%
 Recall : 94.55%

```

F1 Score : 92.18%
ROC AUC : 0.9835
=====

Confusion Matrix

[[14210  2098]
 [ 1079 18717]]

Classification Report

              precision    recall  f1-score   support

     0       0.93       0.87       0.90       16308
     1       0.90       0.95       0.92       19796

 accuracy         0.91
 macro avg         0.91
 weighted avg      0.91

```

5. Leakage Verification

Retrain the model after removing potentially leaking features to verify that performance reflects genuine predictive capability.

```

print(df_cat["Order Status"].value_counts())

print("\n")

print(pd.crosstab(
    df_cat["Order Status"],
    df_cat["Late_delivery_risk"],
    normalize="index"
))

```

```

Order Status
COMPLETE      59491
PENDING_PAYMENT 39832
PROCESSING     21902
PENDING        20227
CLOSED         19616
ON_HOLD        9804
SUSPECTED_FRAUD 4062
CANCELED       3692
PAYMENT_REVIEW 1893
Name: count, dtype: int64

```

```

Late_delivery_risk      0      1
Order Status
CANCELED      1.000000  0.000000
CLOSED        0.433677  0.566323
COMPLETE      0.425140  0.574860
ON_HOLD       0.444104  0.555896
PAYMENT_REVIEW 0.428420  0.571580
PENDING       0.420972  0.579028
PENDING_PAYMENT 0.424533  0.575467
PROCESSING    0.429139  0.570861
SUSPECTED_FRAUD 1.000000  0.000000

```

6. Final Evaluation

Evaluate the validated model using classification metrics and confusion matrix to assess its real-world performance.

```

# =====
# CATBOOST VALIDATION
# Remove Order Status & Product Name
# =====

import joblib

from catboost import CatBoostClassifier

from sklearn.model_selection import train_test_split

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score
)

```